A Memory Management Procedure

This section presents the proposed Online Mean-Feature Sampling (OMFS) algorithm for memory management, as described in Algorithm 2.

Algorithm 2 OMFS: Online Mean-Feature Sampling

Inputs: $\mathcal{B}_u$: Streaming mini-batch, $\mathcal{M}$: Episodic memory with capacity $|\mathcal{M}|_{\max}$, `class_c`: Class counts, `class_m`: Class-wise mean features, θ_f: Encoder network, ϕ: Projection head.
for each $(\texttt{sample}, y) \in \mathcal{B}_u$ **do**
  $\mathbf{f} \leftarrow \phi(\theta_f(\texttt{sample}))$ ▷ Feature vector
  $\mu_y \leftarrow \texttt{class\_m}[y]$
  Update class mean μ_y using Eq. 2
  if $|\mathcal{M}| < |\mathcal{M}|_{\max}$ **then** ▷ $|\mathcal{M}|$: current EM size
    Add $(\texttt{sample}, \mathbf{f}, y)$ to $\mathcal{M}$
    Update class count: $\texttt{class\_c}[y] \leftarrow \texttt{class\_c}[y] + 1$
    Go to next iteration
  end if
  $C_{\text{seen}} \leftarrow$ number of classes with $\texttt{class\_c}[y] > 0$
  $q \leftarrow |\mathcal{M}|_{\max} / C_{\text{seen}}$ ▷ Per-class quota
  $n_y \leftarrow$ number of samples in $\mathcal{M}$ with label y
  if $n_y < q$ **then** ▷ Class under-allocated ⇒ balance priority
    $y_{\max} \leftarrow$ class with largest occupancy in $\mathcal{M}$ (ties broken arbitrarily)
    $(\texttt{sample}^{y_{\max}}, \mathbf{f}^{y_{\max}}, y_{\max}) \leftarrow$ sample of class $y_{\max}$ in $\mathcal{M}$ farthest from $\mu_{y_{\max}}$
    Replace $(\texttt{sample}^{y_{\max}}, \mathbf{f}^{y_{\max}}, y_{\max})$ with $(\texttt{sample}, \mathbf{f}, y)$ in $\mathcal{M}$
    Update class counts:
    $\texttt{class\_c}[y_{\max}] \leftarrow \texttt{class\_c}[y_{\max}] - 1$
    $\texttt{class\_c}[y] \leftarrow \texttt{class\_c}[y] + 1$
  else ▷ Class meets quota ⇒ representativeness priority
    $(\texttt{sample}^{y}, \mathbf{f}^{y}) \leftarrow$ sample of class y in $\mathcal{M}$ farthest from $\texttt{class\_m}[y]$
    if $\|\mathbf{f} - \mu_y\| < \|\mathbf{f}^{y} - \mu_y\|$ **then**
      Replace $(\texttt{sample}^{y}, \mathbf{f}^{y}, y)$ with $(\texttt{sample}, \mathbf{f}, y)$ in $\mathcal{M}$
    end if
  end if
end for

B More Ablation Studies

B.1 Impact of temperature parameter τ

As illustrated in Fig. 5, the performance of TORCH degrades at extreme values of the temperature parameter τ, whereas stable performance is observed for $\tau \in [0.05, 0.2]$.

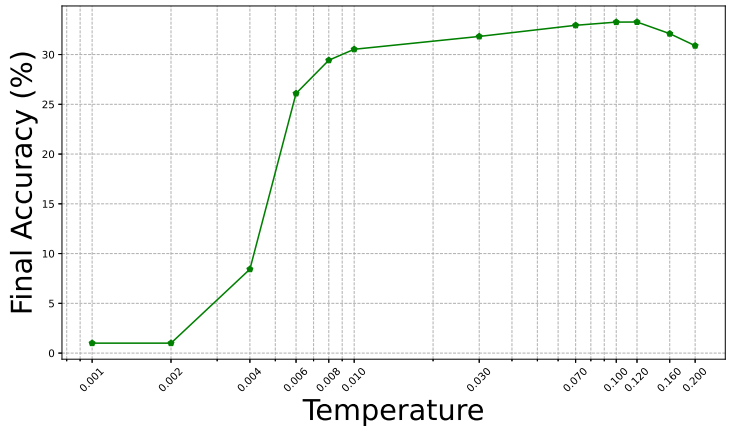

Fig. 5. Sensitivity analysis of the temperature parameter τ on model performance. The results indicate a stable region between 0.05 and 0.2.

Table 3. Effect of different projection head architectures on final accuracy (%) for split CIFAR-100 in the TFOCL setting. Performance remains stable across architectures.

Projection Head	**MLP**	**Linear**	**None**
TORCH	33.7 ± 0.8	32.8 ± 0.9	34.1 ± 0.5

B.2 Impact of Project Network

We evaluate the framework using three configurations: a Multi-Layer Perceptron (MLP), as detailed in Sect. 5.4; a linear layer with 128-dimensional output; and a setup with no projection network (`None`). As presented in Table 3, the inclusion of a projection head yields only marginal performance variances on TFOCL. This indicates that the specific architecture of the projection network has a negligible impact on the overall performance of the proposed method.

B.3 Impact of EM Mini-batch Size during Classifier Training

As discussed in Sect. 4.2, the classifier is trained exclusively using exemplars from the episodic memory (EM). We conduct an ablation study to analyze the effect of the mini-batch size used for classifier updates. Figure 6 illustrates the impact of different mini-batch sizes on overall model performance. We perform ablation studies considering mini-batch sizes $10, 20, 50, 100, 200$. Although a minor performance drop is observed at an EM mini-batch size of 10, performance remains largely stable across the other mini-batch size settings.

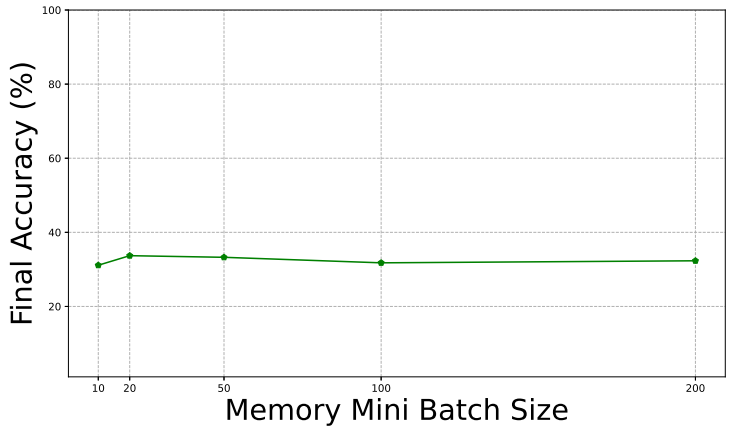

Fig. 6. Impact of EM mini-batch size for classifier training on the split CIFAR-100 dataset ($M = 2000$). Performance remains robust across varying batch sizes.

C Forgetting Metrics

Final Forgetting (FF): *Forgetting* for a task is defined as the difference between the model's peak performance on that task during training and its current performance, estimating how much knowledge has been lost.
Let $a_{i,j}$ denote the accuracy on task i after training up to task j, where $i, j \in \{1, \ldots, N\}$ and N is the total number of tasks. The forgetting for the i-th task after the model has been incrementally trained up to task $k > i$ is denoted by $f_{i,k}$ and can be quantified using the following formula:

$$f_{i,k} = \max_{l \in \{1,\ldots,k-1\}} a_{i,l} - a_{i,k}, \quad \text{for } i < k \tag{6}$$

Here, $f_{i,k} \in [-1, 1]$ measures forgetting for previous tasks i, normalized with respect to the number of tasks seen previously.

The *final forgetting* is then computed as:

$$\mathcal{F}_{\text{final}} = \frac{1}{N-1} \sum_{i=1}^{N-1} f_{i,N} \tag{7}$$

A lower $\mathcal{F}_{\text{final}}$ implies less forgetting of previous tasks.

Forgetting results Table 4 reports the final forgetting (FF) results on split CIFAR-10, split CIFAR-100, and split Mini-ImageNet across different episodic memory (EM) mini-batch sizes. On split CIFAR-10, OBC and PCR exhibit lower FF compared to TORCH. In the case of PCR, the low FF is primarily due to its low initial accuracy (see Fig. 2). For OBC, although the initial accuracy is relatively high, the intermediate task accuracies remain lower than those of TORCH.

Table 4. Final Forgetting (FF) (%) for different datasets and memory sizes. Lower values indicate better performance. Memory size M is expressed in thousands (k).

Methods	split CIFAR-10			split CIFAR-100			split Mini-ImageNet		
	$0.2k$	$0.5k$	$1k$	$0.5k$	$1k$	$2k$	$0.5k$	$1k$	$2k$
Finetune	82.7 ± 1.9	82.7 ± 1.9	82.7 ± 1.9	50.1 ± 1.1	50.1 ± 1.1	50.1 ± 1.1	33.8 ± 0.9	33.8 ± 0.9	33.8 ± 0.9
EWC	83.6 ± 2.5	83.6 ± 2.5	83.6 ± 2.5	48.9 ± 1.0	48.9 ± 1.0	48.9 ± 1.0	32.5 ± 1.5	32.5 ± 1.5	32.5 ± 1.5
ER	53.4 ± 2.6	37.3 ± 3.1	28.7 ± 2.2	33.7 ± 1.2	28.8 ± 1.4	20.3 ± 1.7	28.4 ± 1.5	24.6 ± 1.6	18.2 ± 1.3
DER++	56.8 ± 2.9	43.4 ± 1.6	27.8 ± 1.9	37.2 ± 1.2	32.4 ± 1.0	38.1 ± 1.1	24.7 ± 1.4	22.3 ± 1.2	20.4 ± 1.1
RM	49.8 ± 2.4	36.5 ± 1.8	26.6 ± 1.2	33.9 ± 0.8	26.6 ± 1.0	19.4 ± 1.5	27.5 ± 0.6	20.9 ± 1.0	17.6 ± 1.2
OBC	36.3 ± 2.1	18.4 ± 1.8	17.5 ± 1.1	22.9 ± 1.1	19.0 ± 1.2	10.1 ± 1.0	15.1 ± 1.2	17.8 ± 0.9	12.9 ± 0.8
LAS	51.8 ± 1.6	51.9 ± 1.5	34.6 ± 1.3	29.5 ± 0.9	22.6 ± 1.1	13.3 ± 0.5	25.6 ± 1.0	21.1 ± 1.1	18.5 ± 0.6
CeCR	47.5 ± 2.3	36.1 ± 1.5	27.0 ± 1.2	28.7 ± 1.4	25.2 ± 1.5	20.4 ± 0.6	26.5 ± 1.1	21.6 ± 1.2	19.8 ± 1.1
PCR	33.4 ± 3.1	29.1 ± 1.7	24.0 ± 1.6	24.4 ± 2.9	16.7 ± 1.5	12.8 ± 0.6	23.5 ± 1.0	15.6 ± 1.1	12.9 ± 0.9
TORCH	46.3 ± 2.8	33.4 ± 1.4	24.6 ± 1.1	24.9 ± 1.0	18.9 ± 1.2	11.1 ± 0.8	21.5 ± 1.1	17.0 ± 1.6	12.3 ± 0.9

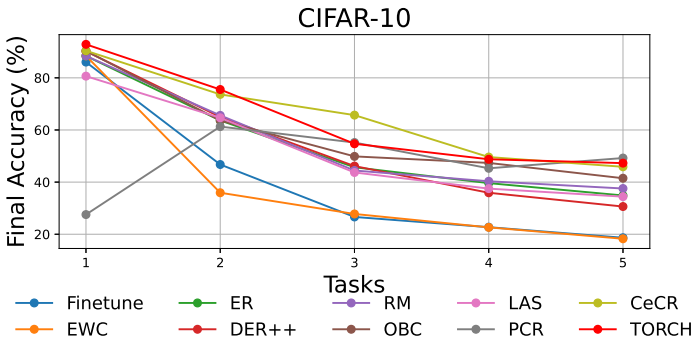

Fig. 7. Final accuracy on split CIFAR-10 ($M = 0.2k$) for different compared methods.

This suggests that the model struggles to learn new information effectively, potentially overfitting on the EM data due to the larger memory mini-batch size relative to the streaming mini-batch size. As a result, OBC shows low forgetting despite having substantially lower final accuracy (FA) and average anytime accuracy (AA) compared to TORCH. On split Mini-ImageNet, however, TORCH consistently outperforms all baselines across all memory settings.

D More Results

The plots in Fig. 7, Fig. 8, Fig. 9, Fig. 10, Fig. 11, and Fig. 12 compare the final accuracy of all evaluated methods across three datasets: split CIFAR-10, split CIFAR-100, and split Mini-ImageNet under various memory settings.

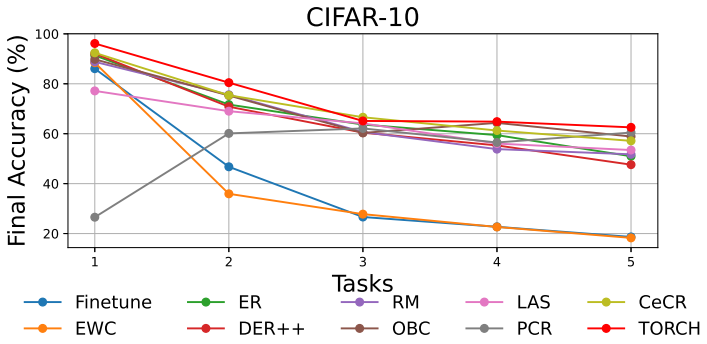

Fig. 8. Final accuracy on split CIFAR-10 ($M = 1k$) for different compared methods.

E Results on Core-50 Dataset

To assess effectiveness of our proposed approach in realistic continual learning settings, we utilize the CORe50 dataset [19] (Lomonaco and Maltoni 2017), a standard benchmark for continuous object recognition. CORe50 contains sequential video frames of 50 objects captured over 11 sessions, each with roughly 300 frames. Consistent with Lomonaco and Maltoni (2017), sessions 3, 7, and 10, comprising about 45,000 images, are reserved for evaluation, while the remaining eight sessions, approximately 120,000 images, form the training stream. This stream is divided into ten tasks, each featuring five objects. Our experiments employ the reduced ResNet-18 architecture described previously, with an EM size of $M = 1k$ and $2k$. Other hyperparameters remain identical to those specified in the main text. We report both the final accuracy (FA), computed over all tasks after training concludes, and the average anytime accuracy (AA), calculated as the mean performance on all classes encountered up to each task throughout the training process. Experimental results (see Table 5) indicate that the proposed method generally outperforms most compared approaches across both evaluation metrics and varying EM sizes. The only exception is for $M = 1k$, where TORCH achieves slightly lower final accuracy than PCR.

F Average Classifier Prediction Bias

To quantify task-wise prediction bias, we define the *average classifier prediction bias* (`Bias`) as the percentage of test samples from old task classes misclassified as new task classes. Let $\mathcal{D}_{\text{old}} = \{(x_i, y_i)\}$ denote test samples from old task classes, and let $\hat{y}_i$ be the predicted label. The `Bias` is computed as:

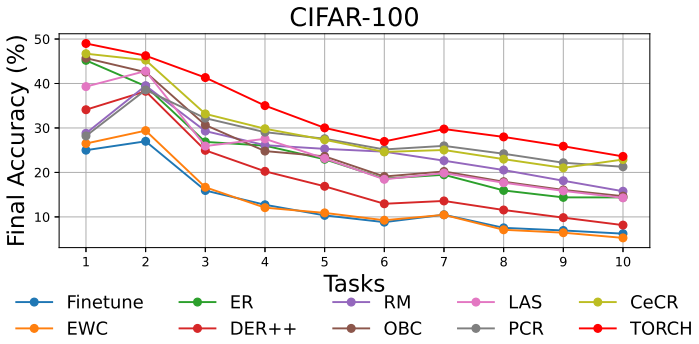

Fig. 9. Final accuracy on split CIFAR-100 ($M = 0.5k$) for different compared methods.

Table 5. Final Accuracy (FA) (%) and Average Anytime Accuracy (AA) (%) for each method on the CORe50 dataset under two memory budgets ($M = 1k$ and $2k$). **Bold** values indicate the best performance for each metric and memory setting.

Method	$M = 1k$		$M = 2k$	
	FA	AA	FA	AA
ER	33.9 ± 2.3	42.8 ± 1.7	37.6 ± 1.8	40.3 ± 1.4
OBC	34.4 ± 1.5	44.8 ± 1.6	38.5 ± 1.2	39.6 ± 2.1
LAS	29.5 ± 0.8	35.4 ± 1.0	33.7 ± 0.9	36.7 ± 1.3
PCR	$\mathbf{38.7 \pm 3.2}$	47.3 ± 2.1	39.9 ± 1.9	49.2 ± 1.8
TORCH	38.3 ± 3.5	$\mathbf{48.7 \pm 2.0}$	$\mathbf{42.8 \pm 2.3}$	$\mathbf{50.6 \pm 1.6}$

$$\texttt{Bias} = \frac{1}{|\mathcal{D}_{\text{old}}|} \sum_{(x_i, y_i) \in \mathcal{D}_{\text{old}}} \mathbb{I}\big(\hat{y}_i \in \mathcal{Y}_{\text{new}}\big) \times 100\%, \tag{8}$$

where $\mathcal{Y}_{\text{new}}$ is the set of labels for new tasks, and $\mathbb{I}(\cdot)$ is the indicator function. This metric provides a direct measure of prediction bias and forgetting. Although our approach does not rely on task boundaries, we hypothesize that this evaluation metric—which leverages task-boundary information—can nonetheless provide valuable insight into the model's performance.

Table 6 presents the average classifier prediction bias (`Bias`) for the compared methods, evaluated on the split CIFAR-100 and split Mini-ImageNet datasets. The results show that TORCH consistently exhibits lower `Bias` across most memory settings, significantly outperforming ER and LAS. PCR performs competitively, particularly under smaller memory budgets. Notably, OBC surpasses

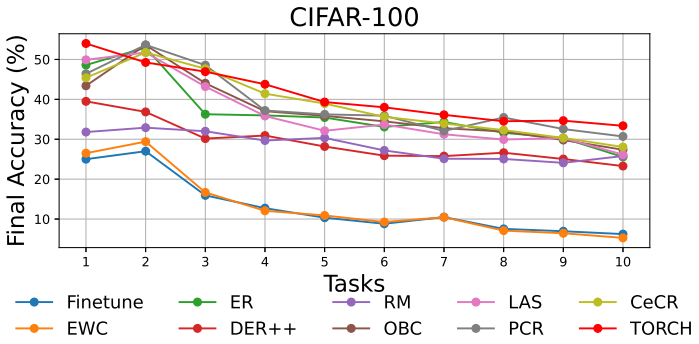

Fig. 10. Final accuracy on split CIFAR-100 ($M = 2k$) for different compared methods.

Table 6. Results showing `Bias` (lower is better) for all compared methods across different memory settings. Values are averaged over 10 runs with a 95% confidence interval.

Methods	**split CIFAR-100**			**split Mini-ImageNet**		
	$M = 0.2k$	$M = 0.5k$	$M = 1k$	$M = 0.5k$	$M = 1k$	$M = 2k$
ER	75.20 ± 3.0	62.61 ± 2.2	48.35 ± 1.9	77.49 ± 2.1	67.81 ± 1.9	49.96 ± 1.3
OBC	37.52 ± 4.1	24.37 ± 2.2	12.96 ± 2.3	27.78 ± 2.9	18.64 ± 3.7	15.77 ± 3.0
LAS	47.11 ± 3.6	32.45 ± 4.0	20.79 ± 3.3	50.87 ± 3.2	34.29 ± 3.8	20.50 ± 4.0
PCR	31.07 ± 3.9	25.32 ± 2.5	20.14 ± 2.2	28.61 ± 2.9	24.00 ± 1.8	24.66 ± 2.0
TORCH	30.43 ± 2.7	23.50 ± 2.1	17.59 ± 1.9	28.56 ± 2.4	22.22 ± 1.7	19.70 ± 1.7

TORCH for larger memory sizes, highlighting its potential as an effective bias-correction approach; however, it performs poorly under smaller memory constraints.

G Motivation behind Anytime Inference

Anytime inference is crucial in continual learning settings because models are often deployed in dynamic environments where training and inference must occur simultaneously. In such scenarios, it is impractical to assume that the model will only be evaluated after completing a predefined training phase. Instead, the model should be capable of producing reliable predictions at any point during training, even as it incrementally observes new data.

For instance, consider an online surveillance or autonomous driving system, where the model continuously encounters new visual patterns while being re-

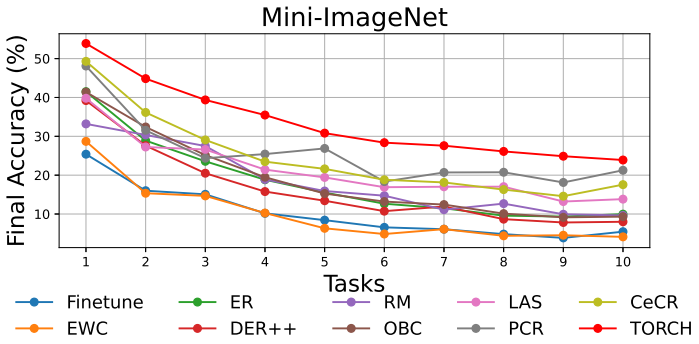

Fig. 11. Final accuracy on split Mini-ImageNet ($M = 0.5k$) for different compared methods.

quired to make real-time decisions. In such applications, pausing the system to retrain a classifier or delaying predictions until training stabilizes is not feasible, as it may lead to critical failures. This requirement is particularly important in real-world applications where decisions must be made in real time and delays in inference can be costly.
Therefore, enabling anytime inference ensures that the model remains usable throughout the learning process, bridging the gap between theoretical performance and practical deployment.

During our literature survey, we observed that contrastive replay-based methods outperform cross-entropy-based baselines. However, contrastive loss operates at the feature representation level and does not directly optimize for classification. Consequently, these methods rely on the Nearest-Class-Mean (NCM) classifier constructed from exemplars stored in episodic memory (EM). As the EM is updated online, class means must be recomputed repeatedly, introducing computational overhead and hindering timely inference.
Alternatively, a separate linear classifier can be used, but it is typically trained only at specific stages, often just prior to evaluation. As a result, both approaches fail to support anytime inference, where predictions are required continuously during training.
These limitations expose a gap between strong representation learning and practical deployment in the TFOCL setting. To address this, we design a training pipeline in which the classifier is trained online alongside the representation model, enabling efficient and reliable anytime inference while retaining the benefits of contrastive learning.

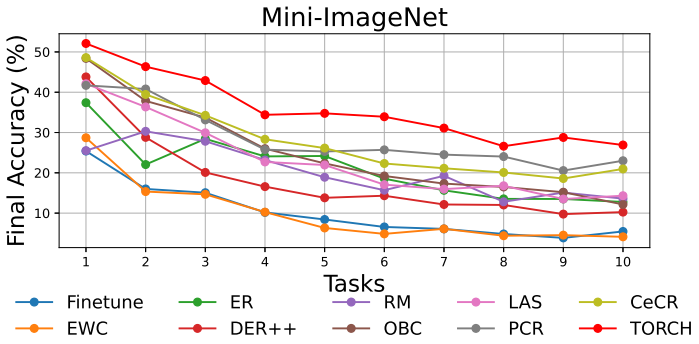

Fig. 12. Final accuracy on split Mini-ImageNet ($M = 1k$) for different compared methods.

H Limitations

Similar to other replay-based continual learning approaches, the proposed method exhibits performance degradation as new classes are introduced. Under a fixed memory budget, the incorporation of additional classes reduces the number of stored samples per class, thereby limiting the representational capacity of the memory. This reduction in per-class exemplars adversely affects the model's ability to retain previously learned knowledge, leading to diminished overall performance.